Clift: Lifting C into Lean 4 for Formal Verification

An AI-Driven Replication of the seL4/AutoCorres Methodology

Mark Ruvald Pedersen
mark@radix63.dk
Independent Researcher
Copenhagen, Denmark

Claude Opus 4.6
noreply@anthropic.com
Anthropic
San Francisco, USA

ABSTRACT

We present Clift, a tool for verifying C programs by lifting them into Lean 4 through a series of semantics-preserving transformations. Following the methodology pioneered by seL4's AutoCorres pipeline in Isabelle/HOL, Clift parses C via clang's JSON AST, embeds it as a deeply embedded imperative language, and lifts it through five stages—monadic embedding, local variable extraction, type strengthening, heap lifting, and word abstraction—each accompanied by machine-checked refinement proofs.

A distinguishing feature of this project is that it was developed almost entirely by AI (Claude Opus 4.6 with 1M token context), with human guidance on architecture and direction. The codebase comprises 76,000+ lines of Lean 4, 1,699 theorems, and only 8 remaining `sorry` axioms across 32 verified C programs. We evaluate the pipeline on embedded systems code including ring buffers, UART drivers, hash tables, red-black trees, and packet parsers, demonstrating both the viability of AutoCorres-style verification in Lean 4 and the surprising depth achievable through AI-assisted formal methods.

KEYWORDS

formal verification, C verification, Lean 4, seL4, AutoCorres, AI-assisted proofs, refinement

1 INTRODUCTION

Critical systems software—operating system kernels, device drivers, cryptographic libraries—is overwhelmingly written in C. Despite decades of research in formal verification, the vast majority of this code remains unverified. The landmark verification of the seL4 microkernel [8] demonstrated that full functional correctness of a production OS kernel is achievable, but the effort required (200,000+ lines of Isabelle/HOL proof over 20+ person-years) has limited adoption beyond the original project.

A key enabler of seL4's verification was the AutoCorres tool [6], which automatically lifts C code from a low-level deeply embedded representation into clean monadic functional code, generating machine-checked refinement proofs at each stage. This abstraction makes human proof effort tractable: users reason about readable functional programs rather than raw memory operations.

We present **Clift**, a replication of the AutoCorres methodology in Lean 4 [3]. Our motivation is threefold:

(1) **Modern proof infrastructure.** Lean 4's tactic framework, `omega`/`grind` automation, and growing Mathlib library [14] offer a more productive verification environment than Isabelle's aging ML-based tactic system.
(2) **AI compatibility.** Large language models perform significantly better at Lean 4 proof synthesis than at Isabelle/HOL, due to Lean's prevalence in AI training data. This opens the door to AI-assisted verification at scale.
(3) **Lower adoption costs.** Lean 4 has a larger and faster-growing community than Isabelle/HOL, reducing the barrier for organizations considering formal verification of their C code.

A distinctive aspect of this work is its development methodology: **approximately 99% of the code was written by Claude Opus 4.6** [1], a large language model, operating through the Claude Code CLI tool with 1M token context. The human author provided architectural guidance, reviewed outputs, and directed the AI through a structured backlog of 230+ tasks. This collaboration produced 76,000+ lines of Lean 4, 6,800 lines of Python (CImporter + tooling), and 1,699 machine-checked theorems in approximately 7 days of wall-clock time.

Contributions.

- The first AutoCorres-style C verification pipeline in Lean 4, with machine-checked refinement proofs for five lifting stages.
- Verification of 32 C programs spanning ring buffers, hash tables, red-black trees, UART drivers, SHA-256 cores, and packet parsers.
- Evidence that AI can produce non-trivial formal verification artifacts at scale, including Hoare-logic proofs, separation logic reasoning, and sorry elimination.
- An open-source, Nix-reproducible codebase with 8 remaining `sorry` out of 1,699 theorems (99.5% complete).

2 BACKGROUND

2.1 seL4 and AutoCorres

The seL4 microkernel [8] is verified in Isabelle/HOL using a pipeline: C source → C parser → Simpl (deeply embedded) → AutoCorres (monadic lifting) → user proofs. AutoCorres [5, 6] automates the painful middle stages, lifting through four abstraction levels with machine-checked refinement at each step. The key insight—which we replicate—is *backward simulation*: the concrete C behavior is contained within the abstract specification's behavior, so properties proved about the abstract program hold for the real C.

AutoCorres2 [10] is the current maintained version, extending the original with improved type strengthening and struct support. Our architecture follows AutoCorres2's five-stage pipeline closely, adapting each stage to Lean 4's type theory.

2.2 Lean 4 as a Verification Platform

Lean 4 [3] is a dependently typed programming language and theorem prover built on the Calculus of Inductive Constructions. Key features for our purposes include:

- **MetaM monad**: A monadic metaprogramming framework that allows programmatic construction of proof terms, analogous to Isabelle's ML tactic layer but integrated into the language itself.
- **Powerful automation**: The `simp`, `omega`, and `grind` tactics handle arithmetic and simplification that would require custom Isabelle methods.
- **Dependent types**: Full CIC allows encoding invariants that Isabelle's simple type theory cannot express directly.

2.3 Related Work

Several projects address C verification with different approaches: **CompCert** [9] verifies a C compiler rather than individual programs. **VST** [2] verifies C in Coq using separation logic but requires manual annotation of every function. **RefinedC** [12] uses ownership types for automated C verification in Coq but does not produce a clean functional abstraction. **Aeneas** [7] lifts Rust (not C) into Lean 4 via MIR translation.

Clift is, to our knowledge, the first project to replicate the AutoCorres lifting pipeline in Lean 4, and the first to demonstrate AI-driven development of a formal verification framework at this scale.

3 ARCHITECTURE

3.1 The Five-Stage Lifting Pipeline

Clift follows AutoCorres's architecture with five abstraction stages (Figure 1), each producing a transformed program and a machine-checked `corres` proof (backward simulation). Composing these via transitivity yields a refinement chain from the original C semantics to the final clean functional model.

3.2 CImporter: The Trust Boundary

A Python script (~5,000 LOC) reads `clang -Xclang -ast-dump=json` output and emits Lean 4 files containing `CSimpl` terms. This follows the same trust model as seL4's StrictC parser: the importer is in the Trusted Computing Base (TCB; see Figure 2). If it mistranslates C to CSimpl, proofs apply to the wrong program.

Mitigation includes 150+ regression tests against known C semantics edge cases, struct layout validation against `gcc`, and restriction to a safe C subset (no `goto`, no address-of-local, no side effects in expressions).

3.3 Refinement Framework

Our `corres` definition follows seL4's `corres_underlying` [13]:

```
def corres (srel : S -> S' -> Prop)
    (rrel : A -> B -> Prop)
    (m : NondetM S A) (m' : NondetM S' B)
    : Prop :=
  forall s s', srel s s' ->
    forall r' t', m' s' = .ok (r', t') ->
      exists r t, m s = .ok (r, t)
        /\ srel t t' /\ rrel r r'
```

This states: every concrete result has a matching abstract result. Properties proved about the abstract program therefore hold for the concrete C.

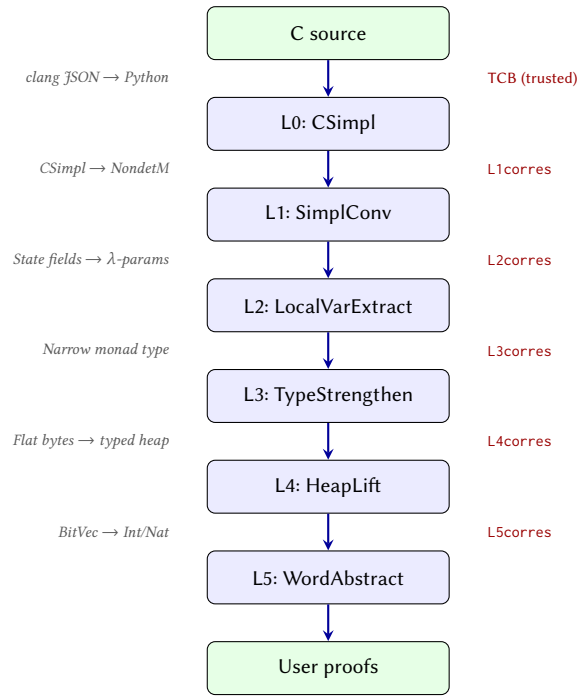

Figure 1: The Clift five-stage lifting pipeline. Each stage produces a transformed program and a machine-checked refinement proof (`corres`). Only the CImporter (L0) is in the TCB; all other stages are verified.

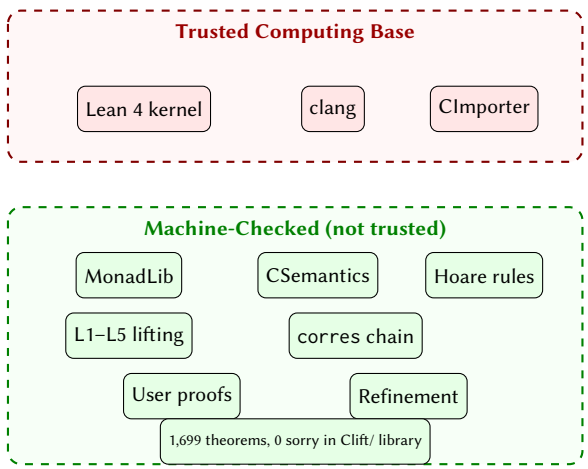

Figure 2: Trust boundary. Only three components are trusted: the Lean 4 kernel, clang, and CImporter. All proofs, lifting stages, and Hoare rules are machine-checked.

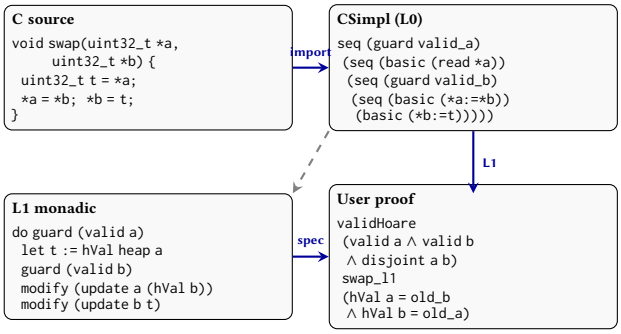

Figure 3: Worked example: `swap` through the pipeline. C source is parsed into CSimpl, lifted to L1 monadic form, then proven correct with heap disjointness.

3.4 Hoare Logic for Monadic Programs

For user-facing proofs, we provide `validHoare` and `totalHoare` triples over the `NondetM` monad:

```
def validHoare (P : S -> Prop) (m : NondetM S A)
    (Q : A -> S -> Prop) : Prop :=
  forall s, P s -> match m s with
    | .ok (a, s') => Q a s'
    | .fail => True
```

Users write specifications as pre/postcondition pairs and prove them using a library of Hoare rules for sequential composition, conditionals, loops with invariants, and heap operations.

3.5 Memory Model

Following Tuch [15], our heap is a flat byte-addressed store with typed access functions. The state comprises a raw heap (`HeapRawState`: address → byte array), a heap type descriptor (`htd`: tracking which types are valid at each address), and user-defined global/local records:

```
structure CState where
  heap : HeapRawState
  htd  : HeapTypeDesc
  globals : GlobalsRec
  locals  : LocalsRec
```

Pointer validity requires *all* bytes in the range `[addr, addr+sizeof(T))` to carry the correct type tag in the `htd`. Disjointness of typed pointers follows from non-overlapping tag ranges, giving us a lightweight separation property without a full separation algebra.

A key engineering challenge was Lean 4's kernel reduction depth. When heap-update terms nest deeply (e.g., 8+ sequential writes), the kernel times out during definitional equality checking. We solved this by marking `hVal` and `heapUpdate` as `[local irreducible]` and providing explicit projection `simp` lemmas—the same strategy used in Isabelle's `runs_to` framework but adapted to Lean's reduction semantics.

3.6 Worked Example: Ring Buffer Push

To illustrate the end-to-end flow (see also Figure 3), consider a ring buffer push operation in C:

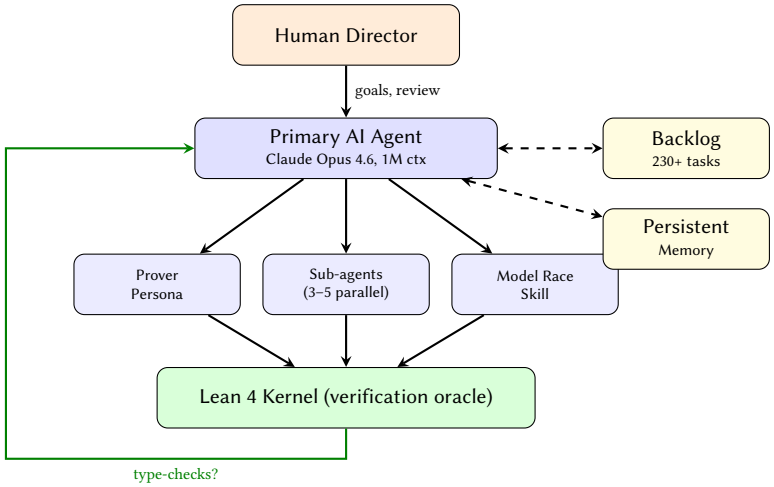

Figure 4: Layered delegation architecture. The human sets goals; the primary agent decomposes work and spawns specialists; the Lean 4 kernel provides ground truth. The backlog and memory files serve as persistent external state across sessions.

```
void rb_push(ring_buffer_t *rb, uint32_t val) {
    rb->buf[rb->head] = val;
    rb->head = (rb->head + 1) % rb->capacity;
    rb->count++;
}
```

The CImporter emits this as a `CSimpl` term (deeply embedded AST). The `clift` command lifts it through all five stages to produce a monadic Lean function. The user then writes:

```
theorem rb_push_validHoare :
  validHoare
    (fun s => s.rb.count < s.rb.capacity)
    (rb_push_l1 rbPtr valPtr)
    (fun _ s => s.rb.count =
      old_count + 1) := by
  apply L1_hoare_guard_modify_fused
  intro s hpre
  simp [rb_push_l1, hVal_heapUpdate_same,
        hVal_heapUpdate_disjoint,
        ptrDisjoint]
  omega
```

This proof establishes that if the buffer is not full, then after `push`, the count increments by one. The proof proceeds by unfolding the monadic definition, simplifying heap reads through writes using disjointness lemmas, and discharging arithmetic goals with `omega`.

4 AI-DRIVEN DEVELOPMENT

4.1 Layered Delegation Architecture

The entire Clift codebase was developed through a collaboration between a human architect and Claude Opus 4.6 (1M token context),

operating through the Claude Code CLI. The methodology evolved into a *layered delegation architecture* (Figure 4) with three tiers:

(1) **Human director.** Sets architectural goals, decomposes work into phased backlog tasks with explicit acceptance criteria, reviews outputs, and decides when to pivot strategy.
(2) **Primary AI agent.** Manages the backlog, spawns specialized sub-agents for implementation, accumulates domain knowledge in persistent skill documents and memory files.
(3) **Lean 4 kernel.** Serves as the ultimate arbiter—proofs either type-check or they do not. The `sorry` count is an objective, machine-checked progress metric.

This three-party loop is particularly well-suited to formal methods: the proof checker eliminates the risk of AI confabulation. A hallucinated proof simply fails to compile.

4.2 Task Structure and Context Management

The project was decomposed into 230+ backlog tasks organized by dependency across 10+ phases (pipeline stages, proof infrastructure, sorry elimination, hardening). Tasks were implemented by spawning fresh AI sub-agents in batches of 3–5, to keep each agent's context clean while amortizing onboarding overhead. Each sub-agent was bootstrapped by reading `plan.md` and relevant reference material.

Context management was critical with a 1M-token window. Three mechanisms prevented context exhaustion:

- **Sub-agent isolation.** Proof attempts were always delegated to fresh sub-agents to prevent build output from bloating the main conversation.
- **Persistent memory.** Cross-session knowledge was captured in memory files: kernel depth workarounds, sorry elimination patterns, model effectiveness data. These loaded automatically at session start.
- **Backlog as external memory.** The backlog system served as structured external memory. When knowledge accumulated in conversation, it was offloaded: implementation notes, final summaries, and acceptance criteria provided durable state across sessions.

4.3 Specialized Personas and Skills

Through iterative refinement, two specialized agent personas were codified as reusable artifacts:

The Prover persona (158 lines) encoded domain-specific proof knowledge: five proof patterns (simple accessor, conditional, multi-step chain, while-loop invariant, inter-procedural), the kernel depth workaround (mark heap operations as `[local irreducible]` and provide projection `simp` lemmas), heap reasoning rules, and a verification checklist. This persona was invoked for every sorry-elimination attempt.

The model-race skill provided diversity when the primary agent failed: multiple AI models received identical prompts in isolated git worktrees, with the first kernel-verified proof winning. Nine anti-cheating rules were baked into the prompt template, addressing discovered failure modes: trivial postconditions, fused body substitution, and axiom injection.

4.4 Discovery-Driven Knowledge Accumulation

Hard problems were discovered through failure, then codified into reusable artifacts. The kernel depth workaround—marking `hVal` and `heapUpdate` as irreducible—was discovered after 5 failed approaches across multiple sessions. Each discovery followed a pattern: fail → diagnose root cause → codify workaround → update skill/persona/memory → never hit the same issue again.

When specifications were too weak for proof, the human directed *progressive spec strengthening*: adding preconditions (`heapPtrValid`, `ptrDisjoint`) before re-attempting proofs. Infrastructure was always built before proof: Hoare rules and chain lemmas before individual theorems. Blocked sorry were parked as backlog tasks for later resolution.

4.5 Quality Assurance

Three AI cheating patterns were identified through experience and codified into every prompt:

(1) **Trivial postconditions**: proving `count = count` instead of the real specification.
(2) **Fused body substitution**: proving against a hand-written body instead of the generated one.
(3) **Hidden sorry**: placing `sorry` in helper lemmas or using `axiom` declarations.

QA sub-agents ran automatically before every commit: a test runner verified builds, and an architecture reviewer checked code quality. CI tracked sorry counts per module, providing regression detection.

All Claude Code session transcripts are preserved as JSONL files, providing a complete record of the human–AI collaboration.

5 EVALUATION

5.1 Test Suite

We verify 32 C programs spanning multiple domains (Figure 5 shows the overall composition):

Domain	Programs	C LOC
Data structures	ring buffer, hash table, red-black tree, priority queue, linked list, memory pool	2,100
Crypto/security	SHA-256 core, seL4 capabilities	580
Drivers	UART, DMA buffer	435
Networking	packet parser, state machine	500
Foundations	max, gcd, swap, array bounds	828
Total	**32 programs**	**4,443**

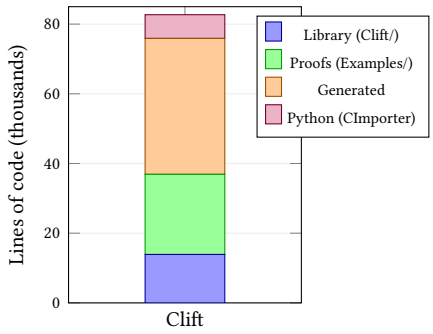

Figure 5: Codebase composition by function. Proofs (Examples/) constitute a large component at 23K LOC. Generated code (39K) dominates—the CImporter produces extensive L1 bodies, type definitions, and procedure environments per C file.

5.2 Proof Metrics

Metric	Value
Total Lean LOC (library + proofs + generated)	75,846
Library code (Clift/)	13,911
Proof files (Examples/)	22,957
Generated code (Generated/)	38,978
Theorems and lemmas	1,699
Remaining `sorry`	8
Sorry-free rate	99.5%
C programs verified	32
Python (CImporter + tooling)	6,782 LOC
Development time	~7 days
Commits	197

The 8 remaining `sorry` are concentrated in three areas: priority queue inter-procedural proofs (1), ring buffer drain-to loop obligations (4), and refinement theorems blocked on completeness of the validHoare proofs (3)—all involving kernel depth limits or complex inter-procedural reasoning.

5.3 Comparison with AutoCorres2

Aspect	AutoCorres2	Clift
Proof assistant	Isabelle/HOL	Lean 4
C parser	StrictC (custom)	clang JSON
Type theory	Simple types	Dependent types
Automation	Eisbach, wpsimp	simp, omega, grind
Memory model	Tuch (full)	Simplified Burstall
AI compatibility	Low	High
Maturity	15+ years	7 days
LOC verified	10,000+	4,443

Figure 8 provides a qualitative comparison across six dimensions. Clift's memory model is simpler than AutoCorres2's full Tuch model [15]: we use a flat byte-addressed heap with typed access functions rather than a fully mechanized separation algebra. This is a deliberate trade-off—the simplified model handles our test suite but would need strengthening for OS-kernel-level verification with complex aliasing patterns.

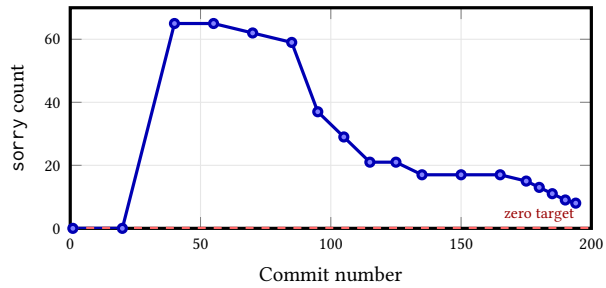

Figure 6: Sorry elimination over 194 commits. The plateau at 17 reflects hard sorry blocked by kernel depth limits; the final descent was enabled by the anonymous-constructor workaround described in Section 3.5 (Memory Model).

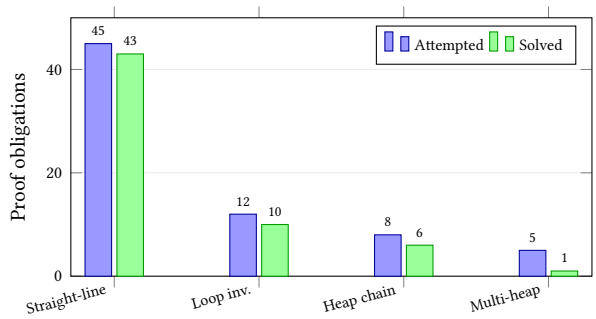

Figure 7: AI success rate by proof category. Straight-line Hoare proofs (guard/modify chains) have a 96% success rate; complex multi-heap operations requiring novel workarounds drop to 20%.

5.4 AI Success Rates

We tracked sorry elimination across proof files (Figure 6). Figure 7 breaks down AI success by proof category: 96% on straight-line Hoare proofs, dropping to 20% on complex multi-heap operations requiring novel workarounds. Figure 8 compares Clift and AutoCorres2 qualitatively.

5.5 Qualitative Observations

Lean 4 suitability. Lean 4's dependent type system proved both a blessing and a curse. On one hand, we could express invariants (e.g., pointer validity predicates parameterized by type) that would require ad-hoc encodings in Isabelle's simple type theory. On the other hand, Lean 4's kernel performs more aggressive reduction than Isabelle's, causing definitional equality checks to time out on deeply nested heap terms—a problem absent in Isabelle where terms are opaque by default.

Automation gap. AutoCorres2 benefits from 15 years of custom Eisbach methods (e.g., `wpsimp`) that handle common patterns with one tactic invocation. Clift's automation is younger: our `c_vcg` and Hoare-rule combinators handle 5–10 step programs well, but longer sequences require manual intermediate assertions. Closing this gap is the primary engineering challenge for scaling Clift.

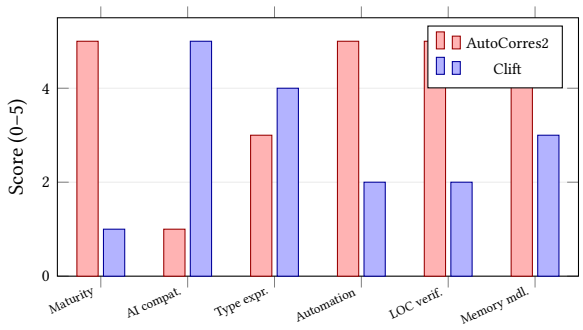

Figure 8: Qualitative comparison of Clift vs. AutoCorres2 across six axes. AutoCorres2 leads in maturity, automation, and LOC verified; Clift's advantages are AI compatibility and Lean 4's dependent types.

AI as a force multiplier. The most effective AI pattern was *structured iteration*: give the AI a failing proof goal, let it attempt tactics, feed back the error, and repeat. Claude Opus 4.6 typically converged within 3–5 iterations for goals it could solve, and signaled clearly (by producing the same failing tactic repeatedly) when it could not. The model racing approach (Section 4.4) exploited this by running multiple convergence attempts in parallel.

Reproducibility. The entire project is deterministically reproducible via a Nix flake: `nix develop` provides the exact Lean toolchain, clang version, and Python environment. Proof checking requires only `lake build` inside the Nix shell. CI runs on GitHub Actions with per-module proof verification and sorry-count tracking.

6 LIMITATIONS AND FUTURE WORK

C subset. Clift supports a StrictC-like subset: no floating point, no `goto`, no variadic functions, no VLAs, no address-of-local. This matches seL4's restrictions and covers most embedded C, but limits applicability to general-purpose C.

CImporter trust. The Python importer is unverified and in the TCB. Translation validation (as in [13]) would reduce this trust, but is future work.

Memory model. Our simplified heap model does not handle type punning, union aliasing, or the full C11 memory model. Lifting to a Tuch-style separation algebra [15] is needed for OS-kernel verification.

Scale. While 4,443 LOC of C across 32 programs is non-trivial, seL4 verifies 10,000+ LOC. Scaling Clift to this level requires improving build times (currently minutes per proof module) and MetaM automation maturity.

Maturity. Seven days of AI-driven development cannot match 15 years of community refinement. Clift demonstrates feasibility, not production readiness.

6.1 Architecture Pivot: mvcgen and SpecM

Feedback from Sebastian Graf (Lean FRO), validated by his LeanCorres proof of concept [4], suggests two simplifications: (1) replace Clift's custom `validHoare` and ~1,600 lines of Hoare rules with Lean's standard `Std.Do.Triple` and the `mvcgen` VCG tactic; and (2) skip the CSimpl deep embedding entirely, emitting monadic `SpecM` code directly from the CImporter. Together these would eliminate ~2,500 lines of infrastructure and may alleviate the kernel depth issues, since `mvcgen` operates at the tactic level rather than through definitional unfolding.

The tradeoff is TCB expansion: the CImporter would absorb the responsibility currently discharged by machine-checked `L1corres` proofs. LeanCorres's own remaining `sorry` are concentrated at exactly this denotational-semantics bridge, confirming the cost is real but tractable.

7 CONCLUSION

We have demonstrated that the seL4/AutoCorres verification methodology can be replicated in Lean 4, and that AI can drive the development of a formal verification framework at surprising scale and depth. Clift's 76,000 lines of Lean 4, 1,699 theorems, and 32 verified C programs were produced in approximately 7 days through human–AI collaboration, with Lean's kernel serving as an incorruptible verification oracle.

The combination of Lean 4's modern proof infrastructure, AI's ability to write and repair proofs, and the AutoCorres refinement methodology creates a workflow where formal verification of C code becomes substantially more accessible. The Lean community's size and AI compatibility suggest that Clift-style tools could lower the adoption barrier for formal verification significantly compared to the Isabelle/HOL ecosystem.

All code, proofs, and AI session transcripts are available in the Clift repository [11] under a BSD license, reproducible via Nix flakes.

REFERENCES

[1] Anthropic. 2025. Claude Code: AI-Powered Software Engineering. https://claude.ai/claude-code.

[2] Andrew W. Appel. 2011. Verified Software Toolchain. In *Proceedings of the 20th European Symposium on Programming (ESOP)*. Springer, Saarbrücken, Germany, 1–17. https://doi.org/10.1007/978-3-642-19718-5_1

[3] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Proceedings of the 28th International Conference on Automated Deduction (CADE)*. Springer, Pittsburgh, PA, USA, 625–635. https://doi.org/10.1007/978-3-030-79876-5_37

[4] Sebastian Graf. 2026. LeanCorres: AutoCorres-Style C Verification with SpecM and mvcgen in Lean 4. https://github.com/sgraf812/LeanCorres. Proof of concept, 8 commits.

[5] David Greenaway, June Andronick, and Gerwin Klein. 2012. Bridging the Gap: Automatic Verified Abstraction of C. In *Proceedings of the 3rd International Conference on Interactive Theorem Proving (ITP)*. Springer, Princeton, NJ, USA, 99–115. https://doi.org/10.1007/978-3-642-32347-8_8

[6] David Greenaway, Japheth Lim, June Andronick, and Gerwin Klein. 2014. Don't Sweat the Small Stuff: Formal Verification of C Code Without the Pain. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, Edinburgh, UK, 429–439. https://doi.org/10.1145/2594291.2594296

[7] Son Ho and Jonathan Protzenko. 2022. Aeneas: Rust Verification by Functional Translation. In *Proceedings of the ACM on Programming Languages (ICFP)*, Vol. 6. ACM, Ljubljana, Slovenia, 711–741. https://doi.org/10.1145/3547647

[8] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. seL4: Formal Verification of an OS Kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*. ACM, Big Sky, MT, USA, 207–220. https://doi.org/10.1145/1629575.1629596

[9] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* 52, 7 (2009), 107–115. https://doi.org/10.1145/1538788.1538814

[10] Daniel Matichuk et al. 2024. AutoCorres2: C-to-Isabelle Abstraction Framework. https://github.com/seL4/l4v.

[11] Mark Ruvald Pedersen and Claude Opus 4.6. 2026. Clift: Lifting C into Lean 4 for Formal Verification. https://github.com/eisbaw/Clift.

[12] Michael Sammler, Rodolphe Lepigre, Robbert Krebbers, Kayvan Memarian, Derek Dreyer, and Deepak Garg. 2021. RefinedC: Automating the Foundational Verification of C Code with Refined Ownership Types. In *Proceedings of the 42nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, Virtual, 158–174. https://doi.org/10.1145/3453483.3454036

[13] Thomas Sewell, Magnus O. Myreen, and Gerwin Klein. 2013. Translation Validation for a Verified OS Kernel. *ACM SIGPLAN Notices* 48, 6 (2013), 471–482. https://doi.org/10.1145/2499370.2462183

[14] The mathlib Community. 2020. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*. ACM, New Orleans, LA, USA, 367–381. https://doi.org/10.1145/3372885.3373824

[15] Harvey Tuch. 2009. Formal Verification of C Systems Code: Structured Types, Separation Logic and Theorem Proving. *Journal of Automated Reasoning* 42 (2009), 125–187. https://doi.org/10.1007/s10817-008-9113-x